Miguel M. Lacanienta
BM4
Sir. John Paul Tomas

# Building Auto-GPT Apps with LangChain

## Installing dependencies and setup

```
[ ] !pip install -U langchain langchain-community faiss-cpu youtube-transcript-api sentence-transformers transformers
    !pip install -U langchain-huggingface
    !pip uninstall bitsandbytes -y
    !pip install --no-cache-dir bitsandbytes
```

```
Requirement already satisfied: jinja2 in /usr/local/lib/python3.11/dist-packages (from torch>=1.11.0->sentence-transformers>=2.6.0->langchain-huggingface) (3.1.6)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch>=1.11.0->sentence-transformers>=2.6.0->langchain-huggingface) (12.4.127)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch>=1.11.0->sentence-transformers>=2.6.0->langchain-huggingface) (12.4.127)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch>=1.11.0->sentence-transformers>=2.6.0->langchain-huggingface) (12.4.127)
Requirement already satisfied: nvidia-cudnn-cu12==9.1.0.70 in /usr/local/lib/python3.11/dist-packages (from torch>=1.11.0->sentence-transformers>=2.6.0->langchain-huggingface) (9.1.0.70)
Requirement already satisfied: nvidia-cublas-cu12==12.4.5.8 in /usr/local/lib/python3.11/dist-packages (from torch>=1.11.0->sentence-transformers>=2.6.0->langchain-huggingface) (12.4.5.8)
Requirement already satisfied: nvidia-cufft-cu12==11.2.1.3 in /usr/local/lib/python3.11/dist-packages (from torch>=1.11.0->sentence-transformers>=2.6.0->langchain-huggingface) (11.2.1.3)
Requirement already satisfied: nvidia-curand-cu12==10.3.5.147 in /usr/local/lib/python3.11/dist-packages (from torch>=1.11.0->sentence-transformers>=2.6.0->langchain-huggingface) (10.3.5.147)
Requirement already satisfied: nvidia-cusolver-cu12==11.6.1.9 in /usr/local/lib/python3.11/dist-packages (from torch>=1.11.0->sentence-transformers>=2.6.0->langchain-huggingface) (11.6.1.9)
Requirement already satisfied: nvidia-cusparse-cu12==12.3.1.170 in /usr/local/lib/python3.11/dist-packages (from torch>=1.11.0->sentence-transformers>=2.6.0->langchain-huggingface) (12.3.1.170)
Requirement already satisfied: nvidia-cusparselt-cu12==0.6.2 in /usr/local/lib/python3.11/dist-packages (from torch>=1.11.0->sentence-transformers>=2.6.0->langchain-huggingface) (0.6.2)
Requirement already satisfied: nvidia-nccl-cu12==2.21.5 in /usr/local/lib/python3.11/dist-packages (from torch>=1.11.0->sentence-transformers>=2.6.0->langchain-huggingface) (2.21.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch>=1.11.0->sentence-transformers>=2.6.0->langchain-huggingface) (12.4.127)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch>=1.11.0->sentence-transformers>=2.6.0->langchain-huggingface) (12.4.127)
Requirement already satisfied: triton==3.2.0 in /usr/local/lib/python3.11/dist-packages (from torch>=1.11.0->sentence-transformers>=2.6.0->langchain-huggingface) (3.2.0)
Requirement already satisfied: sympy==1.13.1 in /usr/local/lib/python3.11/dist-packages (from torch>=1.11.0->sentence-transformers>=2.6.0->langchain-huggingface) (1.13.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from sympy==1.13.1->torch>=1.11.0->sentence-transformers>=2.6.0->langchain-huggingface) (1.3.0)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn->sentence-transformers>=2.6.0->langchain-huggingface) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn->sentence-transformers>=2.6.0->langchain-huggingface) (3.5.0)
Requirement already satisfied: anyio in /usr/local/lib/python3.11/dist-packages (from httpx<1,>=0.23.0->langsmith<0.4,>=0.1.125->langchain-core<0.4.0,>=0.3.15->langchain-huggingface) (3.7.1)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.11/dist-packages (from httpx<1,>=0.23.0->langsmith<0.4,>=0.1.125->langchain-core<0.4.0,>=0.3.15->langchain-huggingface) (1.0.
Requirement already satisfied: h11<0.15,>=0.13 in /usr/local/lib/python3.11/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->langsmith<0.4,>=0.1.125->langchain-core<0.4.0,>=0.3.15->langchain-h
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/dist-packages (from jinja2->torch>=1.11.0->sentence-transformers>=2.6.0->langchain-huggingface) (3.0.2)
Requirement already satisfied: sniffio>=1.1 in /usr/local/lib/python3.11/dist-packages (from anyio->httpx<1,>=0.23.0->langsmith<0.4,>=0.1.125->langchain-core<0.4.0,>=0.3.15->langchain-huggingface)
Downloading langchain_huggingface-0.1.2-py3-none-any.whl (21 kB)
Installing collected packages: langchain-huggingface
Successfully installed langchain-huggingface-0.1.2
WARNING: Skipping bitsandbytes as it is not installed.
Collecting bitsandbytes
  Downloading bitsandbytes-0.45.3-py3-none-manylinux_2_24_x86_64.whl.metadata (5.0 kB)
Requirement already satisfied: torch<3,>=2.0 in /usr/local/lib/python3.11/dist-packages (from bitsandbytes) (2.6.0+cu124)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.11/dist-packages (from bitsandbytes) (1.26.4)
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from torch<3,>=2.0->bitsandbytes) (3.17.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.11/dist-packages (from torch<3,>=2.0->bitsandbytes) (4.12.2)
Requirement already satisfied: networkx in /usr/local/lib/python3.11/dist-packages (from torch<3,>=2.0->bitsandbytes) (3.4.2)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.11/dist-packages (from torch<3,>=2.0->bitsandbytes) (3.1.6)
Requirement already satisfied: fsspec in /usr/local/lib/python3.11/dist-packages (from torch<3,>=2.0->bitsandbytes) (2024.10.0)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch<3,>=2.0->bitsandbytes) (12.4.127)
```

```
[ ] from langchain_community.document_loaders import YoutubeLoader
    from langchain.text_splitter import RecursiveCharacterTextSplitter
    from langchain_huggingface import HuggingFaceEmbeddings, HuggingFaceEndpoint
    from langchain_community.vectorstores import FAISS
    from langchain import PromptTemplate, LLMChain
    import textwrap
    from dotenv import load_dotenv
```

```
[ ] from huggingface_hub import login
    login("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")
```

**Declaring functions**

```
[ ] from youtube_transcript_api import YouTubeTranscriptApi
    import re

    def get_youtube_transcript(video_url):
        """Extracts transcript text from a YouTube video."""
        video_id = re.search(r"v=([a-zA-Z0-9_-]+)", video_url).group(1)
        transcript_list = YouTubeTranscriptApi.get_transcript(video_id)
        transcript_text = " ".join([entry["text"] for entry in transcript_list])
        return transcript_text
```

```
[ ] def create_db_from_youtube_video_url(video_url: str) -> FAISS:
        transcript_text = get_youtube_transcript(video_url)

        text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=100)
        docs = text_splitter.create_documents([transcript_text])

        embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
        db = FAISS.from_documents(docs, embeddings)
        return db
```

```python
import torch
from langchain.schema.runnable import RunnableLambda
from huggingface_hub import InferenceClient
from langchain.prompts import PromptTemplate
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig

def get_response_from_query(db, query, k=4, use_local_model=True):
    """Fetches a response using a local model (if available) or falls back to Hugging Face API."""

    docs = db.similarity_search(query, k=k)
    docs_page_content = " ".join([d.page_content for d in docs])

    # Define the prompt template
    prompt = PromptTemplate(
        input_variables=["question", "docs"],
        template="""
        You are a helpful AI that answers questions based on a YouTube video's transcript.
        Use only the provided transcript and avoid speculation.

        Video Transcript: {docs}
        Question: {question}

        If the answer is unclear, say "I don't know."
        """,
    )

    formatted_prompt = prompt.format(question=query, docs=docs_page_content)

    if use_local_model:
        try:
            # Load local Mistral model with 4-bit quantization
            model_name = "mistralai/Mistral-7B-Instruct-v0.2"

            quant_config = BitsAndBytesConfig(
                load_in_4bit=True,
                bnb_4bit_compute_dtype=torch.float16,
                bnb_4bit_use_double_quant=True,
            )

            tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```python
            model = AutoModelForCausalLM.from_pretrained(
                model_name,
                quantization_config=quant_config,
                device_map="auto"
            )

            # Prepare input and move to GPU
            inputs = tokenizer(formatted_prompt, return_tensors="pt").to("cuda")

            with torch.no_grad():
                output = model.generate(**inputs, max_new_tokens=512)

            response_text = tokenizer.decode(output[0], skip_special_tokens=True)
            return response_text.strip(), docs

        except Exception as e:
            print(f"⚠️ Local model failed: {e}. Falling back to Hugging Face API.")

    # Fallback: Use Hugging Face API if local model fails or is disabled
    try:
        client = InferenceClient(model="mistralai/Mistral-7B-Instruct-v0.2")
        response = client.text_generation(
            formatted_prompt,  # Changed from chat_completion to text_generation
            temperature=0.2,
            max_new_tokens=512
        )
        return response.strip(), docs

    except Exception as e:
        return f"❌ API request failed: {e}", docs
```

## Utilization

```python
[ ] import textwrap

    video_urls = [
        "https://www.youtube.com/watch?v=V8DtMqzN-R0",  # Video 1: Angry Birds
        "https://www.youtube.com/watch?v=jO7s5KhJNPo",  # Video 2: Plants vs. Zombies
        "https://www.youtube.com/watch?v=5e8rA9vL3Xc"   # Video 3: Papa Louie
    ]

    # Ensure dbs are created separately for each video
    dbs = [create_db_from_youtube_video_url(url) for url in video_urls]
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
modules.json: 100%  349/349 [00:00<00:00, 35.6kB/s]
config_sentence_transformers.json: 100%  116/116 [00:00<00:00, 12.7kB/s]
README.md: 100%  10.5k/10.5k [00:00<00:00, 793kB/s]
sentence_bert_config.json: 100%  53.0/53.0 [00:00<00:00, 5.26kB/s]
config.json: 100%  612/612 [00:00<00:00, 66.6kB/s]
model.safetensors: 100%  90.9M/90.9M [00:00<00:00, 188MB/s]
tokenizer_config.json: 100%  350/350 [00:00<00:00, 30.9kB/s]
vocab.txt: 100%  232k/232k [00:00<00:00, 4.43MB/s]
tokenizer.json: 100%  466k/466k [00:00<00:00, 7.49MB/s]
special_tokens_map.json: 100%  112/112 [00:00<00:00, 14.1kB/s]
config.json: 100%  190/190 [00:00<00:00, 444B/s]
```

```python
[ ] # Define queries specific to each video
    video_queries = [
        [  # Queries for Video 1 (Angry Birds)
            "What are the different universes in the Angry Birds franchise?"
        ],
        [  # Queries for Video 2 (Plants vs. Zombies)
            "Why is Crazy Dave crazy?"
        ],
        [  # Queries for Video 3 (Papa Louie)
            "What is the Flipverse?"
        ]
    ]

    # Iterate over each video and its specific queries
    for i, (db, queries) in enumerate(zip(dbs, video_queries)):
        print(f"\n 🔹 **Video {i+1} Results:**\n")
        for query in queries:
            response, _ = get_response_from_query(db, query)

            # Ensure better formatting
            formatted_response = response.replace(". ", ".\n\n")  # Split into paragraphs
            print(f"**Query:** {query}\n\n**Response:**\n{formatted_response}\n{'-'*80}")
```

```
🔹 **Video 1 Results:**

Loading checkpoint shards: 100% ▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰ 3/3 [01:17<00:00, 25.56s/it]
Setting `pad_token_id` to `eos_token_id`:2 for open-end generation.
**Query:** What are the different universes in the Angry Birds franchise?

**Response:**
You are a helpful AI that answers questions based on a YouTube video's transcript.
        Use only the provided transcript and avoid speculation.

        Video Transcript: worth talking about starting with December 11th 2009 Universe 1 the classic Universe this is traditional Angry Birds it's the most popular it has the most stuff within it
        Question: What are the different universes in the Angry Birds franchise?

        If the answer is unclear, say "I don't know."

        Based on the provided transcript, there are at least five major universes in the Angry Birds franchise.

They are:

        1.

December 11th, 2009 Universe 1 (the classic Universe)
        2.

March 22nd, 2012 Universe 2 (the Space Universe)
        3.

Angry Birds Star Wars

        The speaker also mentions a "Joy" universe, but no further information is given about it.

The speaker also mentions that each respective universe is connected to an established universe or its own one, and that they plan to explain how they might connect.

However, no such explanation is given in the transcript.

        Therefore, the answer is: There are at least five major universes in the Angry Birds franchise: December 11th, 2009 Universe 1 (the classic Universe), March 22nd, 2012 Universe 2 (the Spac
```

```
◆ **Video 2 Results:**

Loading checkpoint shards: 100% ▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰ 3/3 [01:18<00:00, 25.92s/it]
Setting `pad_token_id` to `eos_token_id`:2 for open-end generation.
**Query:** Why is Crazy Dave crazy?

**Response:**
You are a helpful AI that answers questions based on a YouTube video's transcript.
        Use only the provided transcript and avoid speculation.

        Video Transcript: if he made the initial Army and then they actually became his ancestors when doing the time travel stuff meaning that he's technically creating himself I guess unlike eve
        Question: Why is Crazy Dave crazy?

        If the answer is unclear, say "I don't know."

        Based on the provided transcript, it is mentioned that Crazy Dave's behavior is either due to experiments that made his brain unhinged or because of him eating food off the ground and gett

However, no definitive reason is given as to why Crazy Dave is crazy.

Therefore, I don't know.

◆ **Video 3 Results:**

Loading checkpoint shards: 100% ▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰ 3/3 [01:16<00:00, 25.29s/it]
Setting `pad_token_id` to `eos_token_id`:2 for open-end generation.
**Query:** What is the Flipverse?

**Response:**
You are a helpful AI that answers questions based on a YouTube video's transcript.
        Use only the provided transcript and avoid speculation.

        Video Transcript: the papa game series has been a beloved collection of flash games for millions of players since the mid 2000s yet for years nobody has tried to figure out how or why thes
        Question: What is the Flipverse?

        If the answer is unclear, say "I don't know."

        The Flipverse is a fictional world in the Papa Louie game series.

It's a land mass of unknown size, large enough to have a modernized society with multiple towns.

The series takes place in this world, and it's been a beloved collection of flash games since the mid 2000s.

The exact nature and size of the Flipverse, including whether it's an island or an entire planet, are not clear from the provided transcript.
```